

Can an AI proof loop replace model checking?

A measured comparison of TLC against machine-closed Lean proofs
with wall-clock and dollar costs

Gavin Mendel-Gleason

Maren Roos

September 27, 2026

Abstract

Model checking a TLA^+ specification with TLC is push-button and exhaustive, but only over a *bounded* instance: it answers “no violation up to N ”. A proof answers “for all N ”, and until recently the cost of obtaining one was human-weeks. We ask whether an AI proof loop — a language model driving Lean 4 through a shell until a mechanically checked closure oracle accepts the artifact — can close that gap, and at what cost.

Across four concurrent algorithms (token-ring mutual exclusion, Lamport’s Bakery, LCR leader election and EWD998 termination detection) the loop closed every task’s general theorem, each with a negative control demonstrating that the rig detects a weakened model. **The headline claim is supported on 4 of 4 tasks: the loop proves the general theorem inside a 2 h / \$50 budget, its worst cell median using 6.2% of the clock and 0.0012% of the money, where TLC cannot answer at any budget.** The same-claim cost comparison is 2–2 and turns on where each instance falls relative to its own crossover rather than on the methods: TLC is faster on two tasks, the loop on two, and cost totals favour the loop on three of four. Every closure cost under a millicent of a dollar, and every proof total — general theorem plus corollary — under 1.5 millidollars.

We report the method, the four tasks, the measured matrix, and at length what the result does *not* establish.

1 Introduction

TLC decides a bounded instance by enumeration. That is the right instrument for a design under development, and its limitation is structural rather than a matter of speed: it cannot establish an invariance for arbitrary N , and its cost grows exponentially in the instance size. Theorem provers answer the general question, but the published human effort is substantial. The TLA^+ Trifecta paper [1] reports **one person-day** for an invariance proof of EWD998, and the IJCAR 2010 TLAPS papers [2] report proof-line counts in the hundreds with no time figure at all.

Three developments make the question worth asking now: reasoning models that can plan proof steps, Lean 4 [3] with Mathlib [4] as a target whose kernel is small and whose libraries are broad, and — the part we emphasise — the ability to *verify* a candidate proof mechanically rather than trust the model that produced it. Recent work measures how often such models can prove benchmark problems [9, 10, 11]. We measure something different: what a *closed* theorem costs in wall-clock and dollars, against the model checker it would replace, on the same specification.

Our question is therefore: *can a loop close general theorems for concurrent systems, at what wall-clock and dollar cost, and how does that compare with what TLC costs on the same claim?*

2 Prior work

Comparing TLC with proof on one specification is not new, and every published comparison pays for the proof in human time. The TLA⁺ Trifecta paper [1] applies TLC, Apalache and TLAPS to EWD998: TLC finds 1.3 million distinct states in 42s at $N = 3$ and 219 million in about 50 minutes at $N = 4$, Apalache checks an inductive invariant at $N = 100$ in 20s, and the TLAPS invariance proof took one person-day and about 230 lines, with safety plus refinement at about 110 and liveness at 245. Pastry [12] is the extreme case: TLC spent more than a month over 1952882411 states without a counterexample, and the authors conclude that interactive proof effort “is too high to scale to more complete P2P protocols”. The IJCAR 2010 TLAPS papers [2] report proof sizes with no time figure at all: Peterson about 130 lines, Bakery 800, Paxos 550 to “somewhat over 1000”. Reports from industry have the same shape: ten AWS systems, specifications of 102–939 lines, two to three weeks of tool learning and 0–3 bugs each [13]; seL4 at about 20 person-years and 480,000 lines of proof [14]. That a model checker settles an instance while a proof settles the theorem is the standard framing of the two methods [15], and it is the framing used here.

What has changed is the price of proof search. Work on AI provers measures how often a model closes benchmark problems [9, 10, 11]; it does not price a closed theorem against the model checker it would replace, on the same specification. **No published work compares an AI-driven proof loop’s wall-clock and dollar cost against TLC’s runtime for the same specification**, and that is the comparison reported below.

The prior art those figures come from is committed here at pinned provenance: the EWD998 specification with both TLAPS proofs (`tlaplus/Examples` at `3dfe0087`, `specs/tla/ewd998/`), the IJCAR 2010 modules (`tlaplus/tlapm` at `7824dab5`, `specs/tla/ijcar2010/`), and the publication-era revision of EWD998 that reproduces the published TLC figures (`specs/tla/ewd998-paper/`). The imported proofs are *not* machine-checked here — `tlapm` is not in the dev shell and is invoked nowhere in this repository — so the effort figures are cited data marked `machine_checked: false`, never a measured arm (§5.3). Two disagreements between the published counts and the committed artifacts are recorded rather than reconciled.

3 Method

3.1 Two routes, two tiers

Route A is TLC enumerating a TLA⁺ specification at a bounded instance. **Route B** is a Lean 4 theorem closed by an AI proof loop.

Each task has two Route B tiers. **Tier 2** is the general theorem: the property for arbitrary N . **Tier 1** is that theorem’s corollary, instantiated at the calibrated instance N_0 . Only tier 1 is comparable with Route A, because only tier 1 proves the same claim.

N_0 is fixed by a stated rule: **the largest N whose TLC run completes inside the 2 h per-run cap**. That definition is what fixes EWD998’s N_0 (§5.1).

3.2 The closure criterion

A *closed* verdict comes from three checks run by code the candidate cannot reach:

1. **Integrity** — the candidate is byte-identical to the seed up to and including the theorem’s `:=`. This pins every definition and the statement; only the proof body is free.
2. **Elaboration** — the candidate is handed to `lean` directly, with no `lake` in the invocation. The candidate has a shell and can reach the package’s lakefile, so a lake-built environment would be *the candidate’s*, not the harness’s.

3. **Closure** — a prebuilt checker imports the elaborated olean and reads the declaration’s axiom set out of Lean’s *API*, against `{propext, Classical.choice, Quot.sound}`.

The third is load-bearing and its implementation matters. `#print axioms` is *syntax*, syntax is an environment extension a candidate can install, and it crosses imports — in testing, a candidate rewrote the command into text of its own and the real report never ran. An API query cannot be reached that way. Both attacks that defeated the first form of this check — a candidate printing a fake axiom report, and a candidate redefining `#print axioms` in its own syntax — were found by an independent reviewer rather than by the loop, and both now fail closed.

Around those three: the bytes checked are one immutable snapshot taken once per round and never re-read, so the verdict, the result row and the durable closure copy all derive from the same bytes; the elaboration environment is captured before the session opens; and the seed’s digest is re-checked on every exit path.

3.3 Why a rate, not a run

Closure is stochastic. The unit of evidence is therefore a **cell**: $R \geq 5$ runs sharing one seed digest, never pooled across configurations. The complement is the **negative control** — the loop is run against the task’s *mutant*, a weakened model, and must *fail* to close it. Without that, a working prover and a broken rig are indistinguishable.

3.4 Measurement

We record wall-clock, peak RSS, dollar cost from provider usage at list price, and distinct states enumerated. Route A runs at its best measured configuration with the row naming it, and worker count is reported because it moves wall-clock but not state counts.

4 The task set

Task	Property	Source
token-ring	mutual exclusion	our bootstrap model
Bakery	mutual exclusion	Lamport [5]
LCR	at most one leader	Le Lann [7], Chang & Roberts [8]
EWD998	termination detection (invariance)	Dijkstra [6]; imported with the proofs of [1]

Table 1: The four tasks. EWD998 is externally calibrated: both the specification and the published TLAPS proofs are imported at pinned provenance, and the human-effort figures attach to it.

5 Results

5.1 The measured matrix

EWD998’s $N_0 = 3$ is the cap’s value rather than a convergence artifact. Its instance search first looked like one: every failing invocation of $N = 4\text{--}6$ carried TLC’s `-cleanup`, which deletes the state pool mid-enumeration, and without it $N=4$ **completes** — 248,006,200 distinct states, depth 104, 2 h 36 min, exit 0. That clean run finishes outside the 2 h per-run cap, so the stated rule of §3.1 selects $N_0 = 3$ on its own authority: removing the flag changed the ground of the value, not the value. The flag is also not a general cause of failure — Bakery reached 238,803,200 distinct states and token-ring 289,406,976 *with* it in place.

	token-ring	Bakery	LCR	EWD998
Route A N_0	23	9	10	3
Route A wall-clock	837.9 s (8 workers)	4,847.7 s	2.7 s	36.4 s
Route A cost	\$0.04655	\$0.26932	\$0.00015	\$0.00202
Route A distinct states	289,406,976	238,803,200	177,147	1,520,618
tier 2 (general N)	closed 104.8 s	closed 444.4 s	closed 216.0 s	closed 162.0 s
tier-2 cost	\$0.00033	\$0.00062	\$0.00044	\$0.00047
tier 1 (corollary)	closed 38.5 s	closed 76.6 s	closed 101.0 s	closed 112.3 s
tier-1 cost	\$0.00023	\$0.00048	\$0.00042	\$0.00051
negative control	TLC violation	fail_to_close	fail_to_close	fail_to_close

Table 2: The measured matrix. Tier-2 figures are medians over the runs that closed, each quoted with its closure count against non-mutant attempts: 6/8, 5/7, 5/5, 5/5 for token-ring, Bakery, LCR and EWD998 (tier 1: 7/8, 5/5, 5/5, 6/6). The five non-closures are three provider faults, one budget timeout, and one unclosed run whose row records no kind; mutant controls are separate runs and never attempts. Ranges are wide — token-ring’s spans 62.9–1021.8 s — so no figure is quoted without its range. Route A at N_0 is a single deterministic run except EWD998, whose six runs give the median shown.

5.2 Same-claim cost, counted honestly

A corollary is the trivial instantiation of its theorem, so the corollary’s cost alone answers “what does instantiation cost *given* the theorem”. The question asked is what *settling the claim* costs, which is theorem plus corollary:

Task	Route A	theorem	corollary	proof total	K	favours
token-ring ($N=23$)	837.938 s	104.8 s	38.5 s	143.4 s	0.171	proof, 5.8×
Bakery ($N=9$)	4,847.749 s	444.4 s	76.6 s	521.0 s	0.107	proof, 9.3×
EWD998 ($N=3$)	36.4 s	162.0 s	112.3 s	274.3 s	7.53	TLC, 7.5×
LCR ($N=10$)	2.670 s	216.0 s	101.0 s	317.0 s	118.7	TLC, 118.7×

Table 3: K = proof ÷ TLC on the same claim. **The split is 2–2.** On cost the totals are kinder to the proof — cheaper on three of four; only LCR favours TLC on both resources.

5.3 Against published human effort

Our task	Human comparator	Reported
ewd998	TLAPS invariance	1 person-day , ~230 lines
ewd998	TLAPS safety + refinement	0.5 person-day , ~110 lines
ewd998	TLAPS liveness	< 1 person-day , 245 lines
ijcar2010-*	TLAPS proofs	lines only — no time reported
token-ring, Bakery, LCR	none published	—

Table 4: Cited data, never a measured arm: the human figures are imported with quotes and pinned provenance, marked `machine_checked: false`, and never pooled with either route.

Only EWD998 has a published person-day figure, so the human comparison is one number against one number: a person-day against EWD998’s **274.3 seconds** of loop time (theorem plus corollary), the widest of the four tasks’ totals being Bakery’s **521.0 seconds**, and every total **under 1.5 millidollars**. The IJCAR papers report lines, and we do not convert lines into days.

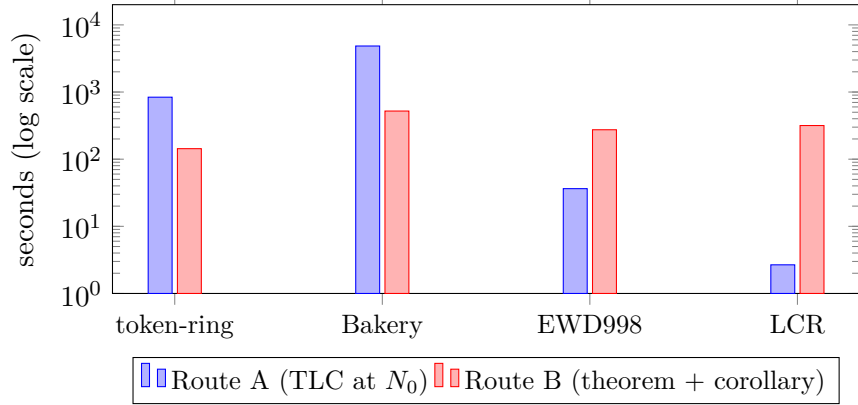


Figure 1: The same claim, both routes, log scale. **The routes trade places rather than one dominating:** the loop is cheaper on token-ring (5.8 $\times$) and Bakery (9.3 $\times$), TLC is cheaper on EWD998 (7.5 $\times$) and LCR (118.7 $\times$). Which way a task falls follows its calibration, not the methods: every N_0 is the instance the rule of §3.1 selected — and for LCR the 2h cap never bound, so the sweep’s extent set it — rather than the instance where the two routes cross. A task calibrated so that both routes are under pressure at the same instance is the experiment this plot shows is still missing.

5.4 The verdict, against each reading of “replacement”

Reading	Result
General theorem inside budget B	Supported, 4 of 4. Worst cell median 444.4s = 6.2% of the 2h cap, and the largest tier-2 cost \$0.00062 = 0.0012% of the \$50 cap. TLC’s coverage of this tier is 0 of 4 at any budget.
Cost parity at the bounded tier (K)	2–2: $K = 0.171 / 0.107 / 7.53 / 118.7$.
Coverage under a fixed budget	Counts equal at 4 of 4, claims not equal: the loop settles four <i>general</i> theorems; TLC settles four <i>bounded</i> instances and zero of the general tier.

Table 5: Protocol readings of what “replacing model checking with proof” might mean, each reported rather than one chosen for the reader.

6 Limitations

These bound the claims, and we would rather state them than have them found.

1. **Different claims per row.** Route A’s row proves a bounded instance; Route B’s tier 2 proves arbitrary N . Only the same-claim pairs of §5.2 compare like with like.
2. **The equivalence is audited, not proved.** Lean-model $\equiv$ TLA⁺-semantics is a hand audit per task. A modelling slip would make the theorem true about the wrong thing.
3. **Model, not code.** Both routes prove properties of models; neither refines to an implementation.
4. **Safety only.** Liveness is out of scope. The published EWD998 figures split on exactly that line (1 person-day invariance, <1 person-day liveness).
5. **No pair was measured at a crossover.** Every N_0 came from the calibration rule of §3.1 — the largest instance Route A completes inside the 2h cap — not from where the

two routes trade places, and for LCR that cap never bound, so the sweep’s extent stands in its place. That is *why* §5.2 splits. The cost shape (TLC exponential, proof flat) is measured; the crossover itself is extrapolated.

6. **Small task set.** Four algorithms, eight settled cells. Not a sampled population.
7. **Different precision across tasks.** Every cell has $R \geq 5$ closures on a single seed digest, but the counts differ — LCR and EWD998 closed 5 of 5 non-mutant attempts, token-ring 6 of 8 (one 2 h timeout), Bakery 5 of 7 (two provider faults) — and the quoted medians are over closures rather than pass rates over attempts. Since EWD998 anchors the human comparison, its 5 of 5 is the one that matters most.
8. **Stochastic costs.** Each row is one sample; verdicts are deterministic given an artifact, costs are distributions. Medians carry ranges for that reason.
9. **Cost basis.** Tokens at list price, compute at a stated host rate, researcher time not costed. Both components are reported separately and never merged.
10. **Our own tooling is young.** The closure oracle is software: the two attacks that defeated its first form (§3.2) were found by review, not by the loop.

7 Conclusion

An AI proof loop closes general theorems for real concurrent algorithms, at a cost of seconds to seventeen minutes and under a millicent, inside a budget it barely touches — the worst cell median uses 6.2% of the 2 h clock and 0.0012% of the \$50. **That answers a question TLC cannot be asked** — arbitrary N rather than a bounded instance — and it does so with a mechanically checked artifact rather than a generated one.

It does not make model checking obsolete. On the same bounded claim, TLC is faster on two of four tasks, and it checks the specification’s semantics directly rather than a ported model whose equivalence is a hand audit. **The honest summary is that the two routes answer different questions at different costs, the crossover between them is the quantity that matters, and no experiment yet measures it.** That is the obvious next step: a task calibrated so that both routes are under pressure at the same instance.

References

- [1] K. Konnov, T. Kuppe, and S. Merz. Specification and Verification With the TLA⁺ Trifecta: TLC, Apalache, and TLAPS. In *ISoLA 2022*. <https://members.loria.fr/SMerz/papers/2022-isola.pdf>
- [2] K. Chaudhuri, D. Doligez, L. Lamport, and S. Merz. Verifying Safety Properties With the TLA⁺ Proof System. In *IJCAR 2010*. <https://members.loria.fr/SMerz/papers/ijcar2010.pdf>
- [3] L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *CADE-28*, 2021.
- [4] The Mathlib Community. The Lean Mathematical Library. In *CPP 2020*.
- [5] L. Lamport. A New Solution of Dijkstra’s Concurrent Programming Problem. *Communications of the ACM*, 17(8), 1974.
- [6] E. W. Dijkstra. Derivation of a Distributed Termination Detection Algorithm. EWD manuscript 998.
- [7] G. Le Lann. Distributed Systems — Towards a Formal Approach. In *IFIP Congress*, 1977.

- [8] E. Chang and R. Roberts. An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes. *Communications of the ACM*, 22(5), 1979.
- [9] DeepSeek-AI. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition. arXiv:2504.21801.
- [10] Numina and Kimi teams. Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning. arXiv:2504.11354.
- [11] Goedel-LM. Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving. arXiv:2502.07640.
- [12] T. Lu, S. Merz, and C. Weidenbach. Towards Verification of the Pastry Protocol using TLA⁺. In *FORTE 2011*. <https://members.loria.fr/SMerz/papers/forte2011pastry.pdf>
- [13] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon Web Services Uses Formal Methods. *Communications of the ACM*, 58(4):66–73, 2015.
- [14] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. seL4: Formal Verification of an OS Kernel. In *SOSP 2009*, pages 207–220.
- [15] E. M. Clarke and J. M. Wing. Formal Methods: State of the Art and Future Directions. *ACM Computing Surveys*, 28(4):626–643, 1996.

A Data provenance

Every figure in §5 is recomputable from rows committed in the repository; none is estimated. The paper builds from `paper/model-or-proof.tex` through the repository’s dev shell (`nix develop -c latexmk -pdf`), so the artifact is produced from its source like everything else here.

Figure	Source
Route A rows	<code>results/tlc.jsonl</code> , with logs under <code>results/logs/</code>
Route B rows	<code>results/proof.jsonl</code> ; closure copies under <code>results/closures/<task>/</code>
Human comparators	<code>results/human.jsonl</code> ; quotes pinned in <code>docs/human-baseline.md</code>
Method and decisions	<code>docs/protocol.md</code>
Closure criterion	<code>harness/closure_oracle.py</code> , <code>tools/checker/PROVENANCE.md</code>
Equivalence audits	<code>docs/equivalence-{token-ring,bakery,lcr,ewd998}.md</code>
Full measured matrix	<code>wiki/comparison-matrix.md</code> , which also carries the two queries that recompute §5.1 and the Route A rows

Table 6: Where each figure comes from. The matrix in `wiki/` exists so the numbers here can be checked against the data rather than trusted as typeset.